

Reproduction of General Graph Random Features

GDL 2025

Group id: —

Project id: —

Weiye Wang, Xingqiao Hu, Zhengtao Gao

{weiye.wang, xingqiao.hu, zhengtao.gao}@usi.ch

Abstract

In this report, we reproduce the General Graph Random Features (g-GRFs) algorithm proposed by Reid et al. [1], which approximates arbitrary functions of weighted adjacency matrices in subquadratic time. The method relies on a modulation function to adjust the weights of random walks based on their lengths. We validate the method’s performance across several tasks, including pointwise estimation of fixed graph kernels, solving graph ordinary differential equations, node clustering, and node attribute prediction using kernels on triangular meshes. Furthermore, we examine the reproducibility of the original experiments and find that some of the reported results cannot be reproduced following the settings in the paper. In several cases, important details are not clearly explained. By adjusting the hyperparameters and making reasonable assumptions about the missing details, we obtain results similar to those reported and confirm the method’s scalability and effectiveness.

1 Introduction

Kernel methods are widely used in machine learning to measure similarity between data points. Graph kernels $K : \mathcal{N} \times \mathcal{N} \rightarrow \mathbb{R}$, defined on the node set $\mathcal{N}$ of a graph $\mathcal{G}$, have diverse applications across various domains. However, these methods often suffer from poor scalability. This is primarily because computing the Gram matrix requires $O(N^3)$ time complexity, with respect to the number of nodes N , due to operations such as matrix inversion or matrix-matrix multiplication involved in the kernel calculation.

To address this, Reid et al. [1] proposed *General Graph Random Features (g-GRFs)*, which yield a pair of feature matrices $\phi_{1,2} := (\phi(i))_{i=1}^N \in \mathbb{R}^{N \times N}$ in subquadratic time, such that $K = \mathbb{E} [\phi_1 \phi_2^\top]$. While obtaining the full Gram matrix K would still require cubic time, most applications can avoid explicit construction by computing matrix-vector products $\phi_1(\phi_2^\top v)$ for any $v \in \mathbb{R}^N$. The core contribution of this work is a *modulation function* that adjusts the weight of random walks based on their length. This mechanism allows the method to approximate arbitrary functions of a weighted adjacency matrix, covering many popular graph kernels. Furthermore, this modulation function can be parameterized by a neural network, offering the possibility to reduce approximation error or to learn a kernel directly from a downstream task.

In this report, we reproduce the main experiments of the proposed method [1]. Beyond standard reproduction, we specifically focused on verifying the experimental configurations. We found that some hyperparameters reported in the original paper failed to yield the presented results, as well as few inconsistencies between the paper and the released code. Consequently, we empirically adjusted these parameters to successfully reproduce comparable results. Our work confirms the effectiveness and efficiency of g-GRFs but highlights the critical importance of careful parameter tuning.

The remaining parts of this report are organized as follows. Section 2 introduces related work. Section 3 presents the

mathematical formulation for constructing g-GRFs. Section 4 details the specific objectives and configurations for each experiment. Finally, Section 5 presents the experimental results and provides a discussion of the findings.

2 Related works

Kernel methods have been widely used for nonlinear learning tasks, as they implicitly map data into a reproducing kernel Hilbert space [2, 3]. In the graph domain, various kernels have been proposed, including diffusion kernels [4], Laplacian kernels [5], and random-walk kernels. These methods have demonstrated success in applications such as graph clustering, node classification, and bioinformatics.

To address the scalability limitations of kernel methods, Rahimi and Recht [6] introduced random features, which approximate kernel functions via Monte-Carlo sampling. Subsequent variants have further improved the efficiency and stability of this approach [7, 8]. However, the application of random features to graph kernels is a more recent development. Choromanski [9] proposed the first Graph Random Feature (GRF) method, which approximates regularized Laplacian kernels by distributing loads across nodes visited by random walkers.

Most recently, Reid et al. [1] generalized this framework by introducing a modulation function to weight random walks based on their length. This extension, termed g-GRFs, overcomes the limitations of prior GRFs by allowing for the unbiased estimation of a broader class of graph kernels. We describe this method in detail in the next section.

3 General graph random features

In this section, we review the theoretical framework of g-GRFs proposed by Reid et al. [1]. We focus on the core mathematical concepts necessary for implementation, omitting exhaustive derivations and proofs, which can be found in the original appendix. We first define the class of graph matrix functions we aim to approximate. Then, we

Table 1. Different graph functions/kernels $K_\alpha^G : N \times N \rightarrow \mathbb{R}$. The exp and cos mappings are defined via Taylor series expansions rather than element-wise. σ and α are regularizers.

Name	Form	Normalized Form	α_k
d -regularized Laplacian	$(\mathbf{I}_N + \sigma^2 \mathbf{L})^{-d}$	$(\mathbf{I} - \frac{\sigma^2}{1+\sigma^2} \widetilde{\mathbf{W}})^{-d}$	$\binom{d+k-1}{k} (1 + \sigma^{-2})^{-k}$
p -step random walk	$(\alpha \mathbf{I}_N - \mathbf{L})^p, \alpha \geq 2$	$(\mathbf{I} + \frac{\widetilde{\mathbf{W}}}{\alpha-1})^p, \alpha \geq 2$	$\binom{p}{k} (\alpha - 1)^{-k}$
Diffusion	$\exp(-\sigma^2 \mathbf{L}/2)$	$\exp(\sigma^2 \widetilde{\mathbf{W}}/2)$	$\frac{1}{k!} \left(\frac{\sigma^2}{2}\right)^k$
Inverse Cosine	$\cos(\mathbf{L}\pi/4)$	$\cos(\widetilde{\mathbf{W}}\pi/4) + \sin(\widetilde{\mathbf{W}}\pi/4)$	$\frac{1}{k!} \left(\frac{\pi}{4}\right)^k \cdot (-1)^{\lfloor k/2 \rfloor}$

describe the random-walk-based estimator and introduce the key concept of the *modulation function*.

3.1 Graph matrix functions

Let $\mathcal{G} = (\mathcal{N}, \mathcal{E}, \mathbf{W})$ be a directed weighted graph with node set $\mathcal{N} = \{1, \dots, N\}$ and weighted adjacency matrix $\mathbf{W} \in \mathbb{R}^{N \times N}$, with w_{ij} the weight of the directed edge from i to j . Now consider the matrix $\mathbf{K}_\alpha(\mathbf{W}) \in \mathbb{R}^{N \times N}$, where $\alpha = (\alpha_k)_{k=0}^\infty$ and $\alpha_k \in \mathbb{R}$:

$$\mathbf{K}_\alpha(\mathbf{W}) = \sum_{k=0}^{\infty} \alpha_k \mathbf{W}^k. \quad (1)$$

With some regularizer $\mathbf{W} \rightarrow \sigma \mathbf{W}, \sigma \in \mathbb{R}_+$, we can assume the sum above converges for all $\mathbf{W}$ under consideration. Without loss of generality, we also assume that α is normalized such that $\alpha_0 = 1$. As pointed by Reid et al. [1], for undirected graphs with symmetric weighted adjacency matrix $\mathbf{W}$, $\mathbf{K}_\alpha(\mathbf{W})$ can be considered the Gram matrix of a graph kernel function K_α^G .

With suitably chosen $\alpha = (\alpha_k)_{k=0}^\infty$, the class described by Equation 1 includes many popular kernels, which measure the connectivity between nodes and typically functions of the *graph Laplacian matrix*, defined as $\mathbf{L} := \mathbf{I} - \widetilde{\mathbf{W}}$ with $\widetilde{\mathbf{W}} := [w_{ij}/\sqrt{d_i d_j}]_{i,j=1}^N$, where $d_i = \sum_j w_{ij}$ is the weighted degree of node i . For reference, in Table 1 we list the kernel definition and their normalized form which can expand into the form describe in Equation 1 using Taylor expansion.

The main goal of this work is to construct random feature maps $\phi(i) : \mathcal{N} \rightarrow \mathbb{R}^l, l \in \mathbb{N}$ such that

$$\mathbf{K}_{ij} = \mathbb{E} [\phi(x_i)^\top \phi(x_j)] \quad (2)$$

provides unbiased approximation of $\mathbf{K}_\alpha(\mathbf{W})$.

3.2 Estimator Construction and Convolution Condition

We construct the random feature vector $\phi_f(i) \in \mathbb{R}^N$ by aggregating m random walks initiated at node i , as detailed in algorithm 1. Intuitively, each walk contributes to the feature vector based on the product of edge weights along the path (the “load”) and a modulation term $f(\text{length})$ that weights the importance of different walk lengths. The final feature $\phi_f(i)$ is the average over these m independent walks.

Given two modulation functions $f_1, f_2 : \mathbb{N} \cup \{0\} \rightarrow \mathbb{R}$, the entries of the target kernel matrix $\mathbf{K}_\alpha$ are approximated

by the inner product of two random feature vector:

$$[\mathbf{K}_\alpha]_{ij} = \mathbb{E} [\phi_{f_1}(i)^\top \phi_{f_2}(j)]. \quad (3)$$

The utility of this construction lies in its theoretical properties. A key result (Theorem 2.1 in [1]) establishes that these random features provide an unbiased estimator for the kernel $\mathbf{K}_\alpha$ if and only if the modulation functions satisfy a discrete convolution constraint. By expanding the expectation of the inner product $\mathbb{E}[\phi_{f_1}(i)^\top \phi_{f_2}(j)]$ and grouping walks by length, one obtains:

$$\mathbb{E}[\phi_{f_1}(i)^\top \phi_{f_2}(j)] = \sum_{k=0}^{\infty} \left(\sum_{p=0}^k f_1(k-p) f_2(p) \right) [W^k]_{ij}. \quad (4)$$

Comparing this expectation with the target kernel definition in Equation 1 reveals the central link between the modulation functions and the kernel coefficients: we achieve equality for all i, j if and only if

$$\alpha_k = \sum_{p=0}^k f_1(k-p) f_2(p), \quad \forall k \geq 0. \quad (5)$$

3.3 Symmetric modulation function

A common and convenient choice is to use the same modulation function on both sides, i.e., $f_1 = f_2 = f$. In this case, the convolution condition simplifies to:

$$\sum_{p=0}^i f(i-p) f(p) = \alpha_i. \quad (6)$$

For $i = 0$, we have $f(0)^2 = \alpha_0$. Assuming the standard normalization $\alpha_0 = 1$, this yields $f(0) = \pm 1$. For $i \geq 0$, isolating the term $f(i+1)$ from the equation for index $i+1$ leads to the following one-step recurrence:

$$f(i+1) = \frac{\alpha_{i+1} - \sum_{p=0}^{i-1} f(i-p) f(p+1)}{2 f(0)}. \quad (7)$$

In practice, Equation 7 allows us to compute $f(i)$ iteratively up to the maximum walk length required for our experiments.

Substituting the convolution condition (Equation 5) back into the definition of $\mathbf{K}_\alpha$ (Equation 1), we obtain:

$$\mathbf{K}_\alpha(\mathbf{W}) = \mathbf{K}_{f_1}(\mathbf{W}) \mathbf{K}_{f_2}(\mathbf{W}), \quad (8)$$

Algorithm 1: Constructing a random feature vector $\phi_f(i) \in \mathbb{R}^N$ to approximate $\mathbf{K}_\alpha(\mathbf{W})$

Input: Weighted adjacency matrix $W \in \mathbb{R}^{N \times N}$; vector of (unweighted) node degrees $d \in \mathbb{R}^N$; modulation function $f : \mathbb{N} \cup \{0\} \rightarrow \mathbb{R}$; termination probability $p_{\text{halt}} \in (0, 1)$; start node $i \in \mathcal{N}$; number of random walks $m \in \mathbb{N}$.

Output: Random feature vector $\phi_f(i) \in \mathbb{R}^N$.

```

 $\phi_f(i) \leftarrow 0$ 
for  $w = 1, \dots, m$  do
  initialize:  $\text{load} \leftarrow 1$ 
  initialize:  $\text{current\_node} \leftarrow i$ 
  initialize:  $\text{terminated} \leftarrow \text{false}$ 
  initialize:  $\text{walk\_length} \leftarrow 0$ 
  while  $\text{terminated} = \text{false}$  do
     $\phi_f(i)[\text{current\_node}] \leftarrow \phi_f(i)[\text{current\_node}] + \text{load} f(\text{walk\_length})$ 
     $\text{walk\_length} \leftarrow \text{walk\_length} + 1$ 
     $\text{new\_node} \leftarrow \text{Unif}(\mathcal{N}(\text{current\_node}))$  // assign to one of the neighbours
     $\text{load} \leftarrow \text{load} \cdot \frac{d[\text{current\_node}]}{1 - p_{\text{halt}}} \cdot W[\text{current\_node}, \text{new\_node}]$  // update load
     $\text{current\_node} \leftarrow \text{new\_node}$ 
     $\text{terminated} \leftarrow (t < p_{\text{halt}})$  where  $t \sim \text{Unif}(0, 1)$  // draw  $t$  to decide on termination
  normalize:  $\phi_f(i) \leftarrow \phi_f(i)/m$ 
return  $\phi_f(i)$ 

```

where $\mathbf{K}_{f_1} := \sum_{i=0}^{\infty} f_1(i) \mathbf{W}^i$. In the symmetric case where $f_1 = f_2 = f$, it follows that:

$$\mathbf{K}_f(\mathbf{W}) = (\mathbf{K}_\alpha(\mathbf{W}))^{\frac{1}{2}}. \quad (9)$$

If $\mathbf{K}_f(\mathbf{W})$ has a simple Taylor expansion, we can derive f directly without relying on the iterative formula in Equation 7. This property holds for many popular kernels, as listed in Table 2.

Table 2. Closed-form modulation functions for popular kernels.

Kernel Name	$f(i)$
d -regularised Laplacian	$\frac{(d-2+2i)!!}{(2i)!!(d-2)!!}$
p -step random walk	$\binom{p/2}{i}$
Diffusion	$\frac{1}{2^i i!}$

4 Implementation

In this section, we present the implementation details of our reproduction study. Our work builds upon the public code repository provided by the authors, which contained a preliminary implementation of the first experiment. However, the provided code yielded results inconsistent with those reported in the original paper. We modified the implementation to enable fine-grained control over hyperparameters, which allowed us to closely reproduce the results presented in the paper. Furthermore, we optimized the computational efficiency of the algorithm, achieving a $\sim 12\times$ speedup, reducing runtime from 195 s to 16 s on an Apple M3 Pro processor for the first experiment.

While the first experiment was optimized based on the authors' code, the remaining experiments were implemented by our team. In the following subsections, we detail the

content, datasets, and hyperparameters for each experiment.

Note: Theoretically, unbiased estimation of diagonal kernel entries (where $i = j$) requires sampling two independent sets of walks to construct separate features ϕ_{f_1} and ϕ_{f_2} . However, in our implementation, we reused a single set of features (i.e., $\phi_{f_1} = \phi_{f_2}$) to compute the Gram matrix. As noted by the authors, this approximation introduces negligible differences in practice while reducing the random walk sampling time by roughly 50%.

4.1 Experiments

4.1.1 Unbiased pointwise estimation of fixed kernels We first verify the capability of g-GRFs to provide unbiased pointwise estimation for the graph kernels listed in Table 1. The original paper states that the approximation quality is measured by the relative Frobenius norm error, defined as $\|K - \hat{K}\|_F / \|K\|_F$. However, the official implementation computes the squared relative Frobenius norm error. To reproduce the results and match the scale of the figures reported in the original paper, we adopted the squared metric $\|K - \hat{K}\|_F^2 / \|K\|_F^2$ in our evaluation. We study the approximation quality represented by the squared relative Frobenius norm error between the exact kernel K and the g-GRF estimate $\hat{K}$ as a function of the number of random walkers m . The approximation using g-GRF utilizes symmetric modulation functions f , computed via closed forms where applicable or the iterative scheme otherwise.

4.1.2 Solving differential equations on graphs An interesting application of g-GRF is to efficiently approximate solutions for time-invariant non-homogeneous ordinary differential equations (ODEs).

Consider the following ODE defined on the nodes $\mathcal{N}$ ((where $|\mathcal{N}| = N$)) of the graph $\mathcal{G}$:

$$\frac{d\mathbf{x}}{dt} = \mathbf{W}\mathbf{x}(t) + \mathbf{y}(t), \quad (10)$$

where $\mathbf{x}(t) \in \mathbb{R}^N$ is the state of graph at time t , $\mathbf{W} \in \mathbb{R}^{N \times N}$ is a weighted adjacency matrix and $\mathbf{y}(t)$ is a known driving term.

Assuming the null initial condition $\mathbf{x}(t) = \mathbf{0}$, Equation 10 is solved by:

$$\mathbf{x}(t) = \int_0^t \exp(\mathbf{W}(t - \tau)) \mathbf{y}(\tau) d\tau. \quad (11)$$

An unbiased Monte Carlo estimator can be constructed as:

$$\hat{\mathbf{x}}(t) := \frac{1}{n} \sum_{j=1}^n \frac{1}{p(\tau_j)} \exp(\mathbf{W}(t - \tau_j)) \mathbf{y}(\tau_j), \quad (12)$$

where $(\tau_j)_{j=1}^n \stackrel{\text{i.i.d.}}{\sim} \mathcal{P}$ are sampled from a distribution with probability density function $p(\tau)$. Note that $\exp(\mathbf{W}(t - \tau_j))$ is in fact the diffusion kernel, which is expensive to compute brute force, but can be efficiently approximated using g-GRF. The approximation quality measures the relative L_2 error of the final state: $\|\hat{\mathbf{x}}(t) - \mathbf{x}(t)\|_2 / \|\mathbf{x}(t)\|_2$.

4.1.3 Efficient kernelized graph node clustering Beyond measuring numerical errors, we further verify if the approximated kernel works well in practice. Specifically, we focus on the task of graph node clustering. We utilized the diffusion kernel $\mathbf{K} = \exp(\sigma^2 \mathbf{W})$ to partition graph nodes into k clusters. It is worth noting that the original paper used the Kernel K-means algorithm described by Dhillon et al. [10]. However, during our reproduction, we observed that the Kernel K-means algorithm typically converged immediately after the initialization using spectral clustering [11] on most of tested graphs. Therefore, we utilized spectral clustering directly to partition the nodes.

The quality of the consistency between the cluster assignments derived from the exact kernel against those obtained via the g-GRF approximation, using the clustering error metric E_c , defined by:

$$E_c := \frac{\text{no. wrong pairs}}{N(N-1)/2}. \quad (13)$$

4.1.4 Learning $f^{(N)}$ for better kernel approximation Instead of relying on a fixed modulation function f derived from specific kernel definition, this experiment introduces a learnable neural modulation function $f^{(N)}$. The primary objective is to investigate whether a learned modulation function can achieve a lower approximation error, measured by the squared relative Frobenius norm $\|\mathbf{K} - \hat{\mathbf{K}}\|_F^2 / \|\mathbf{K}\|_F^2$, compared to the theoretical unbiased estimator. The $f^{(N)}$ was trained to approximate a target kernel on a small synthetic graph and subsequently evaluated to larger, structurally different graphs. This setup allows us to evaluate the transferability of the learned modulation function and its ability to generalize across different graph topologies. Furthermore, we investigated how the number of random walks m affect the learned modulation function.

4.1.5 Implicit kernel learning for node attribute prediction Beyond training the modulation function $f^{(N)}$ to approximate a fixed kernel, it is possible to optimize $f^{(N)}$ directly based on the performance of a downstream task, performing *implicit kernel learning*. Implicitly learning $\mathbf{K}_\alpha$ via $f^{(N)}$ is more scalable than learning $\mathbf{K}_\alpha$ directly because it avoids

repeatedly computing the exact kernel, which is typically of $\mathcal{O}(N^3)$ time complexity.

In this experiment, we performed node attribute prediction on triangular mesh graphs. Each node i is associated with a normal vector $\mathbf{v}^{(i)} \in \mathbb{R}^3$, representing the mean normal of its surrounding faces. The objective is to predict the vectors for a missing subset of nodes (randomly masked) from the known nodes. The unnormalized prediction is given by:

$$\hat{\mathbf{v}}^{(i)} := \sum_{j \in \mathcal{N}_{\text{obs}}} \hat{\mathbf{K}}_{i,j}^{(N)} \mathbf{v}^{(j)}, \quad (14)$$

where $\mathcal{N}_{\text{obs}}$ denotes the set of nodes with known attributes, $\hat{\mathbf{K}}^{(N)}$ is the g-GRF kernel estimate modulated by the learned function $f^{(N)}$. The performance is measured by the angular prediction error, defined as $1 - \cos \theta_i$, where θ_i is the angle between the ground truth vector $\mathbf{v}^{(i)}$ and the prediction $\hat{\mathbf{v}}^{(i)}$.

4.2 Datasets

Our experimental evaluation uses three categories of graph datasets:

- 1. Synthetic Graphs:** We utilized four synthetic graphs: a small ($N = 20, p = 0.2$) and a large ($N = 100, p = 0.04$) Erdős-Rényi graph, a binary tree, and a d-regular graph ($N = 100, d = 10$). These were obtained directly from the authors' code repository.
- 2. Real-World Networks:** Five real-world graphs were used: Karate, Dolphins, Football, Polbooks, and Eurosis, which were obtained from [12].
- 3. Triangular Mesh Graphs:** We used six triangular mesh graphs: Cylinder, Teapot, Idler-riser, Busted, Torus, and Cycloidal, which are available via the Trimesh library.

4.3 Hyperparameters

4.3.1 Unbiased pointwise estimation of fixed kernels The evaluation was performed across eight diverse graph topologies to ensure robustness. These include four synthetic graphs: a small ($N = 20, p = 0.2$) and a large ($N = 100, p = 0.04$) Erdős-Rényi graph, a binary tree, and a d-regular graph ($N = 100, d = 10$). Additionally, four real-world datasets were utilized: Karate, Dolphins, Football, and Eurosis. To maintain consistency with the original paper, we set the regularizer $\sigma = 0.25$ and $\alpha = 20$, with $p_{\text{halt}} = 0.1$.

4.3.2 Solving differential equations on graphs We simulated the diffusion process on three real-world networks: Karate, Dolphins, and Football. We set $\mathbf{W} = -\mathbf{L}$, and using a fixed driving term $\mathbf{y}(t) \equiv \mathbf{y} = (1, 0, 0, \dots)^\top$, which simulates a source at a single node. Although the original paper sets $\mathbf{W} = \mathbf{L}$, we utilized $\mathbf{W} = -\mathbf{L}$ to align with the form of diffusion kernel. The simulation was performed for a duration of $t = 1$. For the integration, we implemented the midpoint rule with $n = 10$ equidistant intervals over $[0, 1]$, rather than the random Monte Carlo sampling described in the Equation 12. We set $p_{\text{halt}} = 0.1$ and the regularizer $\sigma = 1$, consistent with the configuration reported in the original paper.

4.3.3 Efficient kernelized graph node clustering We evaluated the clustering performance on five real-world

graphs: Karate, Dolphins, Polbooks, Football, and Eurosis. Following the original setup, we set the target number of clusters to $k = 3$. The diffusion kernel $\mathbf{K} = \exp(\sigma^2 \mathbf{W})$ was defined using the unweighted adjacency matrix $\mathbf{W}$, with the regularizer set to $\sigma^2 = 0.2$. For the number of random walks m , the original paper only stated an upper bound ($m \leq 80$) without specifying exact values for each graph. We simply set $m = 16$ across all graphs.

4.3.4 Learning $f^{(N)}$ for better kernel approximation We follow the experimental setup described in the original paper. The modulation function was parameterized by a neural network with 1 hidden layer of size 1, defined as:

$$f^{(N)}(x) = \sigma_{\text{softplus}}(w_2 \sigma_{\text{ReLU}}(w_1 x + b_1) + b_2), \quad (15)$$

where $w_1, b_1, w_2, b_2 \in \mathbb{R}$ and σ_{ReLU} and σ_{softplus} are the ReLU [13] and softplus [14] activation functions, respectively. We trained symmetric neural modulation function $f_1^{(N)} = f_2^{(N)} = f^{(N)}$. Training was performed on the small Erdős-Rényi graph ($N = 20$) using $m = 16$ random walks $p_{\text{halt}} = 0.5$ and the regularizer $\sigma = 0.8$, with the 2-regularized Laplacian as the target kernel. We optimized the Frobenius norm loss ($\|\mathbf{K} - \hat{\mathbf{K}}\|_F$) using the Adam [15] optimizer for 1000 epochs. The learning rate was initialized at 0.01 and adjusted using a step scheduler with a decay rate of $\gamma = 0.975$ applied at each epoch. After training, $f^{(N)}$ was used for kernel approximation on other graphs, including larger ER, binary tree, d-regular, Karate, Dolphins, Football, and Eurosis.

To study the effect of the number of random walks, we trained $f^{(N)}$ with $m \in \{2, 4, 8, 16, 32, 64, 128, 256, 512\}$. The original paper did not specify the configuration used for training with different m . In our experiments, we found that a learning rate of 0.1 allowed us to reproduce the trends reported in the paper. To keep the computational cost manageable (especially for large m), we fixed the training duration to 500 epochs, and kept all other hyperparameters the same as in the main setup.

4.3.5 Implicit kernel learning for node attribute prediction This experiment utilized six triangular mesh datasets: Cylinder, Teapot, Idler-riser, Busted, Torus, and Cycloidal. For node attributes prediction task, we randomly masked 5% of the node vectors as the target.

The symmetric neural modulation function $f_1^{(N)} = f_2^{(N)} = f^{(N)}$ was trained on the smallest graph, Cylinder ($N = 210$), by minimizing the angular prediction error. The trained model was then frozen and evaluated on the remaining five larger graphs.

As the specific training hyperparameters were not reported in the original paper, we experimented different configurations to match the reported results. The details are listed below: the Adam [15] optimizer was used with an initial learning rate of 0.01 for 500 epochs. A step learning rate scheduler was applied with a decay factor of $\gamma = 0.99$ per epoch.

For the graph construction, we followed the configuration of the original paper. $\mathbf{W}$ was defined as the normalized weighted adjacency matrix, given by $\mathbf{W} := \mathbf{D}^{-1/2} \mathbf{A} \mathbf{D}^{-1/2}$, where $\mathbf{A}$ is the adjacency matrix and $\mathbf{D}$ is the degree matrix. The number of random walks was set to $m = 16$ and the

halting probability to $p_{\text{halt}} = 0.5$. However, for the scale parameter σ , we were unable to reproduce similar results with $\sigma = 0.025$ reported in the original paper. Consequently, we adjusted the parameter to $\sigma = 0.25$, which produced similar metrics with those reported in the original paper.

4.4 Experimental setup

All experiments were conducted on a MacBook Pro with an Apple M3 Pro processor. The implementation is provided as Jupyter Notebooks and is available in our GitHub repository.

4.5 Computational requirements

All experiments were conducted on a CPU within a reasonable time. The runtime for each experiment is listed in Table 3. Sections 4.1.4 and 4.1.5 include both training and evaluation; we report the neural network training time here. In Section 4.1.4, we also conducted an experiment to study how the number of random walks m affects the learned modulation function. The training times for different values of m are listed in Table 4. For the evaluation in Section 4.1.5, we report the evaluation time on different graphs in Table 5.

Table 3. Runtime for each experiment.

Experiment	Runtime
4.1.1	~ 16 s
4.1.2	~ 27 s
4.1.3	~ 3 s
4.1.4 (training)	~ 20 s
4.1.5 (training)	~ 2 min

Table 4. Training time for different m .

m	Training Time
2	~ 1 s
4	~ 2 s
8	~ 5 s
16	~ 10 s
32	~ 20 s
64	~ 40 s
128	~ 1 min 26 s
256	~ 2 min 56 s
512	~ 5 min 53 s

Table 5. Evaluation time on different triangular meshes.

Graph Name	N	Evaluation Time
Cylinder	210	~ 6 s
Teapot	480	~ 14 s
Idler_riser	782	~ 24 s
Busted	1941	~ 1 min
Torus	4350	~ 3 min 50 s
Cycloidal	21384	~ 4 min 30 s

5 Results

5.1 Unbiased pointwise estimation of fixed kernels

Figure 1 illustrates the unbiased estimation of popular kernels on graphs of varying sizes using g-GRFs. Each

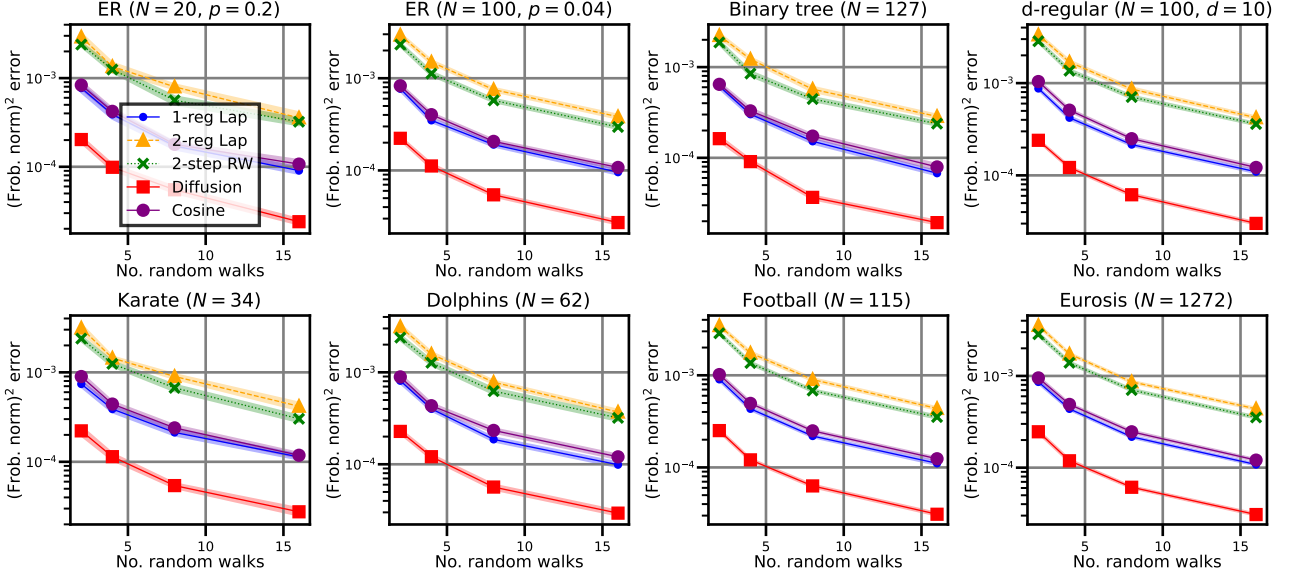


Figure 1. Unbiased estimation of popular kernels on various graphs using g-GRFs. The approximation error (squared relative Frobenius norm) decreases as the number of random walks m increases.

experiment was repeated 10 times. The shaded region represents one standard error of the mean ($\sigma/\sqrt{N}$). We can observe that for all graphs and kernels, the squared relative Frobenius norm of the g-GRF approximation decreases as the number of random walks m increases. This confirms the ability of g-GRFs to provide unbiased pointwise estimation of fixed kernels. We successfully reproduced the convergence trends and error values for all kernels with the exception of the inverse cosine kernel, as the value of regularizer was not specified in the original paper.

5.2 Solving differential equations on graphs

The relative L_2 norm error of the ODE simulation, $\|\hat{x}(t) - x(t)\|_2 / \|x(t)\|_2$, is plotted against the number of random walks m in Figure 2. Statistics were calculated over 100 independent runs, with the shaded region representing one standard error of the mean. Similar to the pointwise kernel estimation results, the approximation quality improves and the error decreases for every graph as m grows. Although the absolute error values differ from those reported in the original paper because we simulate a different diffusion process, the convergence trend is consistent. This result further supports the effectiveness of g-GRFs.

5.3 Efficient kernelized graph node clustering

The clustering error metric E_c , defined in Equation 13, represents the proportion of misclassified node pairs (i.e., pairs assigned to the same cluster when they should be distinct, or vice versa) relative to the total number of pairs. The E_c results for 5 graphs of varying sizes are presented in Table 6. Experiments were repeated 10 times, and we report the mean error $\pm$ one standard error of the mean. We observe that the error remains low even on large graphs using a modest number of random walks ($m = 16$). This demonstrates that kernels estimated via g-GRFs achieve performance comparable to exact methods while being much more computationally efficient.

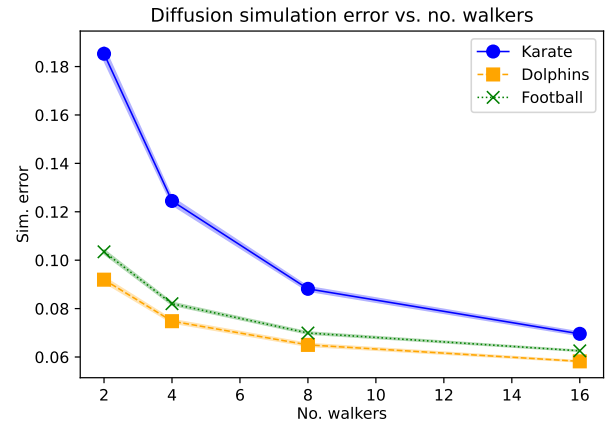


Figure 2. ODE simulation error decreases as the number of walkers grows.

Table 6. Errors in spectral clustering using the kernel $\exp(\sigma^2 \mathbf{W})$ approximated by g-GRFs.

Network	N	Clustering error (E_c)
Karate	34	0.1027 ± 0.0320
Dolphins	62	0.1240 ± 0.0360
Polbooks	105	0.0557 ± 0.0099
Football	115	0.0563 ± 0.0066
Eurosis	1,272	0.0050 ± 0.0008

5.4 Learning $f^{(N)}$ for better kernel approximation

Table 8 presents the performance of the learned function $f^{(N)}$, trained on the Small ER graph, when applied to kernel approximation on other graphs. Experiments were repeated 10 times, and we report the mean error $\pm$ one standard error of the mean. We observe that the learned $f^{(N)}$ consistently yields lower error compared to the unbiased f when using $m = 16$ random walks.

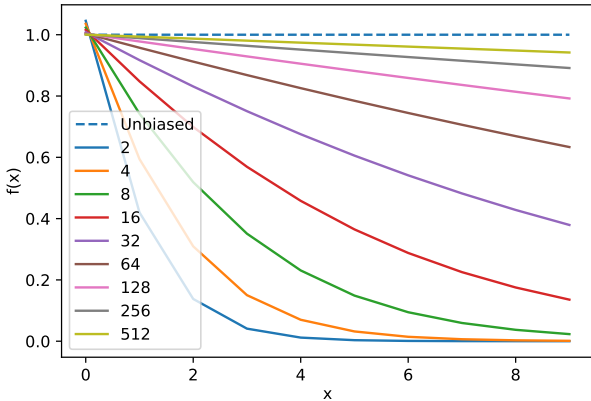
Table 7. Normalized difference in angular prediction error compared to the learned kernel defined by $f^{(N)}$ (trained on the Cylinder graph)

Mesh Model	N	1-reg Laplacian	2-reg Laplacian	Diffusion
Cylinder	210	0.0690 ± 0.0086	0.1349 ± 0.0141	0.0130 ± 0.0017
Teapot	480	0.1148 ± 0.0080	0.2336 ± 0.0134	0.0251 ± 0.0015
Idler Riser	782	0.0708 ± 0.0051	0.1439 ± 0.0084	0.0146 ± 0.0011
Busted	1,941	0.0853 ± 0.0046	0.1688 ± 0.0075	0.0160 ± 0.0009
Torus	4,350	0.0234 ± 0.0038	0.1319 ± 0.0059	0.0059 ± 0.0007
Cycloidal	21,384	0.0604 ± 0.0022	0.1223 ± 0.0034	0.0108 ± 0.0004

A key finding from this experiment is that $f^{(N)}$ does not converge to the unique unbiased (symmetric) modulation function implied by α . Instead, it converges to a different function that, although biased, achieves a smaller overall error. The original paper explains this phenomenon to the bias-variance tradeoff. For a small m , the learned $f^{(N)}$ gives estimators with a smaller variance by downweighting long walks. Therefore, it is intuitive that as m grows, the variance on the kernel approximation drops, so the learned $f^{(N)}$ will approach the unbiased f as shown in Figure 3.

Table 8. Kernel approximation error with $m = 16$ walks with learned and unbiased modulation functions.

Graph Type	N	Squared Frob. norm error on $\hat{K}$	
		Learned Error	Unbiased Error
Small ER	20	0.0360 ± 0.0016	0.0504 ± 0.0029
Large ER	100	0.0334 ± 0.0007	0.0478 ± 0.0011
Binary Tree	127	0.0357 ± 0.0011	0.0433 ± 0.0020
d -regular	100	0.0366 ± 0.0004	0.0486 ± 0.0007
Karate	34	0.0365 ± 0.0019	0.0517 ± 0.0031
Dolphins	62	0.0352 ± 0.0010	0.0488 ± 0.0026
Football	115	0.0364 ± 0.0004	0.0518 ± 0.0010
Eurosis	1,272	0.0369 ± 0.0003	0.0552 ± 0.0007

**Figure 3.** Learned modulation function with different numbers of random walkers m . It approaches the unbiased $f^{(N)}$ as m grows.

5.5 Implicit kernel learning for node attribute prediction

To evaluate the quality of the kernel $\hat{K}^{(N)}$ implicitly defined by the learned modulation function $f^{(N)}$, we computed the angular prediction error on several larger graphs and compared it against modulation functions corresponding to

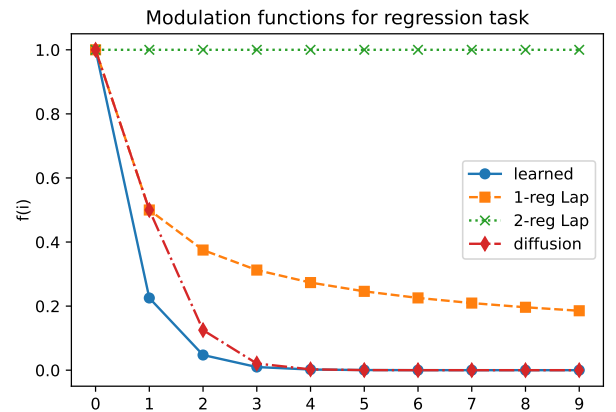
popular fixed kernels. Figure 4 visualizes these modulation functions over the domain $\{0, \dots, 9\}$.

It is important to note that absolute prediction errors are highly correlated with the quality of the sampled random walks. If the random walks fail to adequately explore the graph, all modulation functions will yield poor g-GRF estimators. To reduce this variance, Table 7 reports the average normalized difference relative to the learned kernel’s performance, denoted as $\Delta(\text{pred error})$. Although the exact formula was not explicitly provided in the original paper, we inferred the following formulation as the most plausible interpretation based on the textual description:

$$\Delta(\text{pred error}) = \frac{\text{Error}(\hat{K}^{\text{fix}}) - \text{Error}(\hat{K}^{(N)})}{\text{Error}(\hat{K}^{(N)})}. \quad (16)$$

The results are summarized in Table 7. Experiments were repeated 100 times (with the exception of the large Cycloidal graph, which used 10 repeats), and we report the mean $\pm$ one standard error of the mean. The metric is consistently positive across all graphs and fixed kernels, indicating that the learned kernel consistently performs best.

Remarkably, the learned $f^{(N)}$ achieves the lowest error on all graphs, despite being trained only on the Cylinder mesh, which is the smallest graph in the dataset. This suggests that we have implicitly learned a high-quality kernel that generalizes robustly across different topologies. It is also interesting that the diffusion kernel performs only slightly worse than the learned kernel in this task. This is expected, as their modulation functions are similar (Figure 4) and therefore encode similar kernels.

**Figure 4.** Fixed and learned modulation functions to approximate kernel for node attribute prediction task.

6 Discussion and conclusion

In this reproduction study, we reproduced *General Graph Random Features* (g-GRFs), a random walk-based algorithm proposed by Reid et al. [1] for the time-efficient estimation of arbitrary functions of weighted adjacency matrices. Our experiments validate both the effectiveness and computational efficiency of g-GRFs. The algorithm is conceptually straightforward and trivially parallelizable, unlocking kernel-based machine learning on very large graphs. Furthermore, by parameterizing the modulation component of the random features with a simple neural network, we confirmed that g-GRFs can further suppress the error of estimators and enable scalable implicit kernel learning.

We also implemented a benchmark experiment included in the appendix of the original paper, which measures the wall-clock time for exact and approximate kernel evaluation as the number of nodes increases. However, we did not report the results because our machine did not have enough memory to handle the computation on $\mathbb{R}^{12800 \times 12800}$ adjacency matrix required for the largest setting. This explicitly demonstrates the scalability constraints that g-GRFs is designed to overcome.

While the algorithm is effective and efficient, the reproduction process itself is challenging. In particular, we observed a inconsistency between the error metric definitions in the paper and that in the code repository in Experiment 4.1.1. Additionally, the normalized form of kernels in Table 1, which should have been explicitly provided based on Equation 1, were missing, forcing us to derive them manually and making the implementation more difficult. Moreover, several hyperparameters were unspecified in Experiments 4.1.1, 4.1.4, and 4.1.5, and some metric definitions were unclear (e.g., in Experiment 4.1.5). Most notably, using the configurations reported in the paper failed to reproduce the stated results for Experiment 4.1.5, necessitating empirical adjustments. Despite these documentation gaps, the core methodology remains robust and valuable.

References

- [1] Isaac Reid, Krzysztof Choromanski, Eli Berger, and Adrian Weller. General graph random features. In *International conference on learning representations*. OpenReview, 2024.
- [2] Alexander J Smola and Bernhard Schölkopf. Bayesian kernel methods. In *Advanced lectures on machine learning*, pages 65–117. Springer, 2002.
- [3] Colin Campbell. Kernel methods: A survey of current techniques. *Neurocomputing*, 48(1–4):63–84, 2002.
- [4] Risi Kondor and John Lafferty. Diffusion kernels on graphs and other discrete input spaces. In *International conference on machine learning*, pages 315–322. Morgan Kaufmann, 2002.
- [5] Alexander J Smola and Risi Kondor. Kernels and regularization on graphs. In *Conference on learning theory*, pages 144–158. Springer, 2003.
- [6] Ali Rahimi and Benjamin Recht. Random features for large-scale kernel machines. In *Advances in neural information processing systems*, pages 1177–1184. NeurIPS, 2007.
- [7] Krzysztof Choromanski, Mark Rowland, and Adrian Weller. The unreasonable effectiveness of structured random orthogonal embeddings. In *Advances in neural information processing systems*, pages 1–12. NeurIPS, 2017.
- [8] Felix Xinnan Yu, Ananda Theertha Suresh, Krzysztof Choromanski, Daniel Holtmann-Rice, and Sanjiv Kumar. Orthogonal random features. In *Advances in neural information processing systems*, pages 1–12. NeurIPS, 2016.
- [9] Krzysztof Marcin Choromanski. Taming graph kernels with random features. In *International conference on machine learning*, pages 5964–5977. PMLR, 2023.
- [10] Inderjit S. Dhillon, Yuqiang Guan, and Brian Kulis. Kernel k-means: spectral clustering and normalized cuts. In *Proceedings of the Tenth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, KDD '04*, page 551–556, New York, NY, USA, 2004. Association for Computing Machinery. ISBN 1581138881.
- [11] Jianbo Shi and J. Malik. Normalized cuts and image segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 22(8):888–905, 2000.
- [12] Vladimir Ivashkin and Pavel Chebotarev. Do logarithmic proximity measures outperform plain ones in graph clustering? In *International Conference on Network Analysis*, pages 87–105. Springer, 2016.
- [13] Vinod Nair and Geoffrey E. Hinton. Rectified linear units improve restricted boltzmann machines. In *Proceedings of the 27th International Conference on International Conference on Machine Learning, ICML'10*, page 807–814, Madison, WI, USA, 2010. Omnipress.
- [14] Charles Dugas, Yoshua Bengio, François Bélisle, Claude Nadeau, and René Garcia. Incorporating second-order functional knowledge for better option pricing. In T. Leen, T. Dietterich, and V. Tresp, editors, *Advances in Neural Information Processing Systems*, volume 13. MIT Press, 2000.
- [15] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.